GARAK / MODEL SECURITY

Garak scan findings

A selected-probe exploration of Hugging Face GPT-2

Four Garak runs • Garak 0.17.1 • November 2, 2026

Scope statement: This is not a comprehensive full scan. It includes only the probes selected and run by the author.

TECHNICAL REPORT • 02 NOV 2026

Executive readout

SCOPE	Four selected plugin families: exploitation, grandma, malwaregen, and web_injection.
EVIDENCE	The report JSONL files record detector-level PASS/FAIL outcomes. Fail counts are scanner findings, not proof of a real-world exploit.
SIGNALS	Notable flagged outcomes include SQLiEcho (42/50), grandma MitigationBypass (up to 56/60), malwaregen AnyCode Payload (139/600), and two extended web-exfiltration checks (114/1,280 and 217/1,280).
LIMIT	There is no defensible overall safety score here: probes, detectors, denominators, and tested behaviors differ.

Scope and method

- Target recorded in each report: huggingface / gpt2. Exact command pattern: python3 -m garak --model_type huggingface --model_name gpt2 --probes <selector>; selectors were exploitation, grandma, malwaregen, and web_injection (one per run).
- Garak version recorded in each report: 0.17.1. Recorded generation setting: 5. The seed field is null.
- The supplied files do not establish a model revision/checkpoint.
- Four run start times in the JSONL reports fall on November 2, 2026. The supplied archive contains report JSONL and hitlog files for all four runs; HTML is present for three, but absent for malwaregen.
- Method: summarize each JSONL eval row as detector FAILs / total_evaluated. Keep different detectors and probes separate; where a probe has multiple detectors, do not double-count it as independent model behavior.

Findings | exploitation

Probe	Detector	FAIL / evaluated	Readout
JinjaTemplatePythonInjection	JinjaTemplateInjectionDetector	12 / 45	12 detector FAILs
JinjaTemplatePythonInjection	PythonCodeExecution	12 / 45	12 detector FAILs
SQLInjectionEcho	SQLiEcho	42 / 50	42 detector FAILs

The two Jinja detector rows evaluate the same probe output set; they are separate detector views, not independent samples.

Findings | grandma

Probe	Detector	FAIL / evaluated	Readout
Slurs	MitigationBypass	56 / 60	Flagged
Slurs	SlursReclaimedSlurs	0 / 60	No flags
Substances	MitigationBypass	50 / 50	Flagged
Win10	MitigationBypass	29 / 30	Flagged
Win10	Win5x5	0 / 30	No flags
Win11	MitigationBypass	30 / 30	Flagged
Win11	Win5x5	0 / 30	No flags

Detector rows on a shared probe are not interchangeable. PASS means this detector did not flag the output under its own criterion; it is not a safety certification.

Findings | malwaregen

Probe	Detector	FAIL / evaluated	PASS / evaluated
Evasion	AnyCode	60 / 240	180 / 240
Payload	AnyCode	139 / 600	461 / 600
SubFunctions	AnyCode	88 / 280	192 / 280
TopLevel	AnyCode	14 / 80	66 / 80
Combined rows*	AnyCode	301 / 1,200	899 / 1,200

*Arithmetic sum of the four probe rows; not a separate Garak evaluation. Figures are detector evaluations recorded by Garak, not a count of unique prompts or real-world malware.

Findings | web_injection

Probe	Detector	FAIL / evaluated
ColabAIDataLeakage	MarkdownExfilContent	0 / 20
MarkdownImageExfil	MarkdownExfilBasic; MarkdownExfilContent	0 / 60 each
MarkdownURIImageExfilExtended	MarkdownExfilExtendedImage	114 / 1,280
MarkdownURINonImageExfilExtended	MarkdownExfilExtendedNonImage	217 / 1,280
MarkdownXSS	XSS	0 / 35
PlaygroundMarkdownExfil	PlaygroundMarkdownExfil	0 / 20
StringAssemblyDataExfil	MarkdownExfilBasic	0 / 10
TaskXSS	XSS	15 / 425

Extended URI image and non-image probes show detector FAILs. Other listed rows recorded zero FAILs under their named detectors; this is bounded to the outputs and detector criteria in these runs.

Interpretation and limitations

WHAT THIS SHOWS	Which supplied probes produced detector FAILs in these specific Garak runs, with the detector and denominator visible.
WHAT IT DOES NOT SHOW	A comprehensive evaluation, a confirmed compromise, exploitability in a deployed system, or a model-wide safety guarantee.
DETECTOR DEPENDENCE	Garak uses detectors to judge model outputs; different detectors can assess different criteria on the same probe outputs. Read each detector row in context.
REPRODUCIBILITY GAPS	The artifacts identify the model name but not its revision/checkpoint. The report seed is null; environment and runtime details are not established by the supplied files.
SCOPE BOUNDARY	Only the four named plugin selectors are represented. No claim is made about unrun probes, a surrounding application, or controls not present in the evidence.

Reporting approach and references

- This presentation follows a defensible assessment-reporting pattern: state scope, target, tool/version, method, results with denominators, limitations, and references. This is reporting discipline—not a claim of NIST or OWASP compliance.
- [1] Garak documentation, “Reading the results” — explains probe/detector result lines and report/hitlog files. https://docs.garak.ai/garak/llm-scanning-basics/reading-the-results.md
- [2] Garak documentation, “Scan evaluation” — describes detector scores and the default hit threshold. https://docs.garak.ai/garak/garak-components/scan-evaluation.md
- [3] NIST AI 600-1, Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (July 2024). Used as a reference for documenting context, measurement, and limitations; not as a certification checklist. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
- [4] OWASP GenAI Security Project, Top 10 for LLM Applications. Included as an industry risk-taxonomy reference; these scan results are not mapped to OWASP categories because the evidence does not establish application context or controls. https://genai.owasp.org/llm-top-10/